

C++ 1일차 보고서

1. 서론: 학습 내용 및 보고서 개요

오늘 C++ 1일차 교육을 통해 C++의 핵심 개념과 객체지향 프로그래밍의 기초를 학습했다. 이번 보고서에서는 오늘 다룬 문법과 개념들을 단순히 요약하는 것을 넘어, 지능형 로봇 팀 인턴으로서 배우고 있는 로봇 제어 프로그래밍에 이 개념들을 어떻게 써먹을 수 있을지 중심으로 내 생각을 정리해 보았다. 아울러 오늘 수행한 과제(HW1, HW2)를 어떤 의도로 설계하고 코드를 작성했는지도 함께 담았다.

2. C++ 핵심 개념 탐구 및 깨달음

2.1 Namespace의 실무적 접근

C++에서 namespace는 함수나 변수 이름이 겹치는 걸 막아주는 기능이다. 보통 코딩할 때 `using namespace std;`를 많이 쓰는데, 지능형 로봇처럼 센서 제어나 모터 구동 등 여러 모듈을 합쳐야 하는 프로젝트에서는 이름 충돌이 버그로 이어질 수 있다. 따라서 좀 귀찮더라도 `std::cout`처럼 명시해주거나, 내가 개발하는 파트 코드를 따로 namespace로 묶어서 관리하는 방어적인 코딩 습관이 필수적일 것 같다.

2.2 직관적인 입출력 스트림 (cin, cout)

C언어에서 `printf`나 `scanf`를 쓸 때는 서식 지정자를 일일이 맞춰야 해서 꽤 번거롭고 실수하기도 쉬웠다. 그런데 C++의 `cout`, `cin`은 `<<`, `>>` 연산자만으로 데이터 타입에 맞춰서 알아서 처리해주니 훨씬 직관적이었다. 타입이 안 맞아서 생기는 오류를 알아서 막아주니까, 테스트 중 센서 데이터를 빠르게 확인하고 디버깅할 때 훨씬 안정적이고 편할 것 같다.

2.3 참조자(Reference)와 포인터의 차이

오늘 배운 것 중 제일 유용하다는 생각이 들었던 건 참조자다. C언어의 포인터는 강력하긴 하지만, 로봇이 주행하다가 주소 오류로 코드가 뺏어버리면 시스템 전체가 멈추는 문제가 발생할 수 있다. 반면 참조자는 선언할 때 무조건 초기화해야 하고, 가리키는 대상을 중간에 바꿀 수 없어서 훨씬 안전하다. 덩치 큰 센서 데이터를 함수로 넘길 때, 포인터 대신 참조자를 쓰면 속도는 빠르면서도 메모리 훼손 위험 없이 데이터를 다룰 수 있을 것 같다.

2.4 절차지향에서 객체지향(OOP)으로

C언어는 데이터를 정의하고 그걸 순서대로 처리하는 느낌이었다면, C++은 데이터와 함수를 하나의 '객체'로 묶어서 관리한다는 점이 신선했다. 특히 `private`이나 `public` 같은 접근 지정자로 외부에서 함부로 센서 값이나 모터 상태 변수를 건드리지 못하게 막을 수

있다는 점이 좋았다. 데이터가 꼬이는 걸 원천적으로 차단하고 정해진 함수로만 접근하게 만드니까, 코드가 훨씬 단단해지는 느낌이다.

3. 객체지향 프로그래밍의 로봇 제어 접목 고찰

이런 객체 지향적인 생각은 지능형 로봇 하드웨어를 제어할 때 유용할 것 같다. 예를 들어 로봇의 '모터 구동부'나 '거리 측정 센서'를 각각 하나의 클래스로 만드는 거다. 모터 클래스를 만든다면 '현재 온도'나 '최대 속도 한계 값' 같은 중요한 변수들은 무조건 private으로 숨겨서 외부 제어 루프에서 실수로 값을 덮어쓰지 못하게 안전장치를 걸어 둔다. 대신 set_speed() 같은 public 함수를 만들어서, 그 안에서 한계 속도를 넘는지 검사하는 로직을 거치게 하면, 하드웨어 과부하를 예방할 수 있을 것이다. 또 처음에 통신 포트를 여는 작업은 '생성자'에, 프로그램이 끝날 때 안전하게 모터 전원을 차단하는 건 '소멸자'에 딱 넣어두면, 시스템의 시작과 끝을 훨씬 깔끔하게 관리할 수 있다.

4. 결론 및 향후 코드 작성 방향

오늘 C++ 1일차 수업을 들으면서, 단순히 코드를 순서대로 짜는 걸 넘어서 '어떻게 구조를 튼튼하게 짤까'를 고민하게 되었다. 앞으로 코딩할 때는 다음 세 가지를 꼭 지켜보려고 한다.

1. 설계 먼저 하기: 무작정 키보드부터 잡지 말고, 모터나 센서 같은 기능들을 클래스로 따로 빼서 묶을지 밑그림을 먼저 그릴 생각이다.
2. 참조자 적극 쓰기: 영상 데이터나 큰 구조체를 넘길 때는 포인터 대신 const 참조자(&)를 써서 처리 속도도 높이고 오류도 방지할 것이다.
3. 파일 분할과 정보 숨기기: 헤더랑 소스 파일은 무조건 분리하고, 중요한 제어 변수는 일단 private으로 닫아두고 꼭 필요한 것만 public으로 열어주는 코딩 습관을 들여야겠다.

5. 금일 과제(HW1, HW2) 코드 설계 및 적용 과정

앞서 고민하고 4.결론에서 세운 '코드 작성 방향(설계 먼저 하기, 파일 분할 및 정보 숨기기 등)'을 오늘 과제를 수행하며 실제로 어떤 과정으로 적용했는지 정리해 보았다.

5.1 '설계 먼저 하기'와 '정보 숨기기' 적용

무작정 main 함수부터 코딩을 시작하던 습관을 버리고, HW1(배열 계산기)과 HW2(좌표 거리 계산기)를 각각 어떤 클래스로 만들지 밑그림부터 그렸다. 클래스의 핵심 변수(HW1의 원소 배열, HW2의 2차원 좌표 데이터 등)는 무조건 private으로 닫아두었다. 대신 배열의 최대/최소/합계/평균을 구하거나 두 점 사이의 유클리디안 거리를 계산하는 기능들만 public 멤버 함수로 열어두어, 객체 생성 후 직관적이고 안전하게 호출할 수 있

도록 설계했다.

5.2 '파일 분할'의 생활화

파일 하나에 모든 코드를 때려 넣지 않고, 클래스의 뼈대가 되는 선언부(.h)와 실제 동작이 들어가는 구현부(.cpp)를 철저히 분리했다. 이렇게 나누어 작성해보니 가독성도 훨씬 좋아졌고, 나중에 지능형 로봇의 수많은 모듈을 코딩할 때 어떻게 파일을 분리해서 관리해야 할지 감을 잡을 수 있었다.

5.3 동적 할당, 오버로딩 등 C++ 특징 적극 활용

- **생성자/소멸자를 통한 메모리 관리:** 사용자가 입력한 개수만큼 생성자에서 배열을 동적으로 할당(new)하고, 객체가 수명을 다할 때 소멸자가 알아서 해제(delete)하도록 만들어 메모리 누수를 방지했다.
- **오버로딩을 통한 사용성 개선:** HW2에서 결과를 출력할 때, 단순히 좌표만 찍는 함수와 최대/최소 정보까지 포함해 찍는 함수를 같은 이름(print_result)으로 '오버로딩'해 보았다. 함수 이름 하나로 상황에 맞게 골라서 작동하도록 만드니, API를 사용하는 입장에서 훨씬 편리했다. 이렇게 얻은 깨달음을 바탕으로 향후 참조자(&) 등의 문법도 적극 적용해 나갈 계획이다.